The Islamia University of Bahawalpur

Online jewelry Store

A project presented to

Department of Information Technology, IUB Bahawalpur

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Infromation Technology (2022-2026)

By

Arooba

F21BINFT1E02045 2022-2026

DECLARATION

I hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that I have developed this software and accompanied report entirely on the basis of my personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, I will stand by the consequences. No portion of the work presented has been submitted for any other degree or qualification of this or any other university or institute of learning.

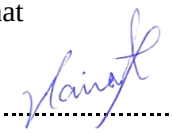
Arooba

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (IT) "Online Jewelry Store" was developed by Arooba, F21BINFT1E02045, SESSION 2026 under the supervision of "Miss Kainat" and that in her opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Information Technology.

Supervisor

Miss Kainat



.....

External Examiner

.....

Chairman Department of Information Technology

Dr. Omer Riaz .....

Acknowledgement

This project represents the culmination of my own research, effort and dedication. I am proud to have brought it to fruition through HTML, CSS, JavaScript, Python with Django and SQLite — independent research, exploration of various concepts, and applying problem-solving techniques.

I am deeply grateful to my supervisor Miss Kainat for her continuous guidance, valuable feedback, and unwavering support throughout the development of this project. Her expertise and encouragement were instrumental in shaping this work.

I also extend my sincere thanks to the Department of Information Technology, The Islamia University of Bahawalpur, for providing the resources and environment necessary to complete this project.

Student Name

Arooba

Abbreviations

Abbreviation	Full Form
SRS	Software Requirements Specification
SDD	Software Design Document
IT	Information Technology
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
DB	Database
UI	User Interface
API	Application Programming Interface
CRUD	Create, Read, Update, Delete

Table of Contents

1 Introduction

The Online Jewelry Store is a user-centric e-commerce platform designed to facilitate seamless online shopping for jewelry products. Users can browse, search, and purchase fine jewelry, fashion accessories, and handcrafted pieces securely. The platform includes both a customer-facing storefront and an admin panel for managing products, orders, and customer data. The website prioritizes an intuitive interface, efficient order processing, and adherence to security standards.

This project is built using HTML, CSS, and JavaScript for the frontend, Python with Django for the backend, and SQLite as the database. It aims to provide a visually appealing, responsive, and fully functional online shopping experience tailored for the Pakistani jewelry market.

1.1 Brief

The Online Jewelry Store enables customers to register, search for products, manage a shopping cart, and place secure orders. Administrators can manage product listings, view orders, and handle customer accounts. The system is designed with a focus on reliability, usability, and security.

1.2 Relevance to Course Modules

This project applies knowledge from multiple course modules. HTML, CSS, and JavaScript (studied in Web Technologies) form the frontend. Python with Django (studied in Web Engineering and Software Development) handles the backend. Database concepts from the Database Management Systems course are applied using SQLite. Software Engineering principles guide the architecture and lifecycle of the project.

1.3 Project Background

The jewelry retail industry in Pakistan is large and growing, yet most local jewelers lack a digital presence. Customers often travel long distances to physical stores with limited selections. This project addresses that gap by providing a comprehensive online platform where customers can explore and purchase jewelry from anywhere, at any time.

Key motivations for this project include:

- Wider Selection: Customers can browse diverse jewelry categories including gold, silver, gemstones, and fashion accessories.
- Convenience: 24/7 shopping access from any device, eliminating time and geographic constraints.

- Accessibility: Reaching customers in remote areas of Pakistan who lack access to quality jewelry stores.
- Digital Transformation: Helping local jewelers move their business online with a modern, professional platform.

1.4 Related Material and Literature

Research indicates that user experience (UX) and security are critical for e-commerce success. Platforms like Daraz.pk, Amazon, and Etsy demonstrate best practices for product presentation, search, and checkout flows. Studies on Pakistani e-commerce highlight mobile responsiveness and localized payment methods as key success factors. Django's documented MVC (Model-View-Template) pattern provides a proven framework for rapid, secure web development.

1.5 Analysis from Literature Review (Online Jewelry Store)

The literature review for this project provided valuable insights into trends and best practices in the online retail jewelry sector:

1.5. 1 Importance of User Experience (UX) and User Interface (UI)

Jewelry is a visual product — customers need high-quality images, zoom functionality, and detailed descriptions. Studies emphasize that intuitive navigation and a smooth checkout process are crucial for conversion rates.

1.5. 2 Mobile Responsiveness

A significant portion of Pakistani online shoppers use mobile devices. The platform must be fully responsive and optimized for small screens.

1.5. 3 Security and Trust

Online jewelry purchases involve higher transaction values. Secure authentication, HTTPS, and clear return policies are essential for building customer trust.

1.6 Methodology and Software Lifecycle for This Project

The Agile methodology was selected for this project due to its iterative and incremental nature. Development was broken into sprints: requirements gathering, design, frontend development, backend integration, testing, and deployment. This allowed for continuous feedback and adaptation throughout the project lifecycle.

1.6. 1 Rationale behind Selected Methodology

Agile was chosen because:

- It supports flexible, iterative development suitable for a web application with evolving requirements.
- It allows the student to deliver working features incrementally and test them with real users.
- Risk management is easier as issues are identified and resolved per sprint rather than at the end.
- Continuous improvement through reflection after each iteration leads to a higher-quality final product.

2 Problem Definition

The rapid growth of digital commerce has changed consumer behavior globally, including in Pakistan. Traditional brick-and-mortar jewelry stores face increasing competition from online platforms, yet many existing online solutions are outdated, insecure, or lack the features needed for a high-quality jewelry shopping experience.

2.1 Problem Statement

Local jewelry shoppers in Pakistan face limited options for online purchasing. Existing platforms either do not support local jewelers or provide poor user experiences with slow navigation, lack of product detail, and insecure payment methods. On the administrative side, jewelry store owners struggle to manage inventory and orders without a proper digital system.

This project addresses these issues by developing a fully functional, user-friendly, and secure Online Jewelry Store built with Django and SQLite.

2.2 Deliverables and Development Requirements

Deliverables

- Functional E-commerce Website: A fully operational jewelry store allowing users to browse, search, add to cart, and checkout.
- Administrator Panel: Backend system for managing products, orders, and customers.
- Database System: Structured SQLite database storing users, products, orders, and transactions.
- Design Documents: SRS, SDD, Use Case Diagrams, Class Diagrams, and Data Flow Diagrams.
- Testing Reports: Documentation of functional, usability, and security testing outcomes.
- User Manual: Guide for customers and administrators.

Development Requirements

Frontend Development:

- HTML5
- CSS3 (with Bootstrap)
- JavaScript (Vanilla)

Backend Development:

- Python (Version 3.10 or higher)
- Django (Version 4.x)

Database:

- SQLite (Django default, Version 3.x)

Development Tools:

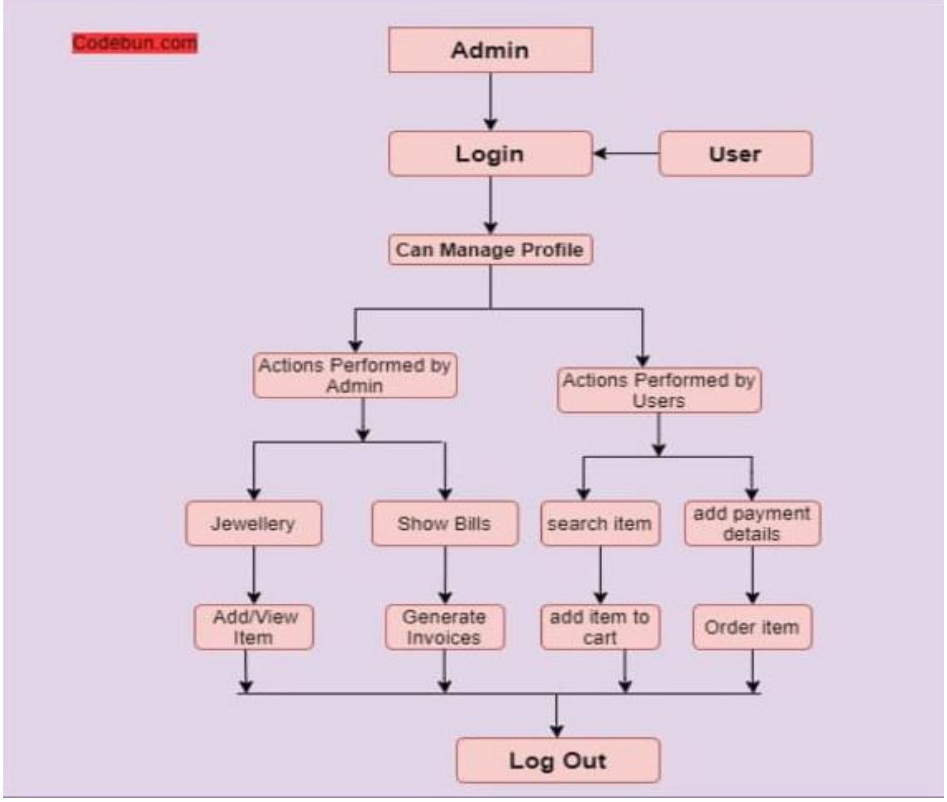
- Visual Studio Code
- Git for version control
- Django Admin Panel
- Browser testing (Chrome, Firefox, Edge)

3 Requirement Analysis

3.1 Use Cases Diagram

A use case diagram summarizes the interactions between actors (User and Admin) and the system. The main actors are:

- User: Registers, logs in, searches products, adds to cart, and places orders.
- Admin: Manages product listings, orders, and customer accounts.



[Figure 1: Use Case Diagram — Online Jewelry Store]

Use Case ID	Use Case Name	Actor	Description
UC-1	User Registration	User	User creates a new account with unique email and password.
UC-2	User Login	User	User authenticates with valid credentials.
UC-3	Browse Products	User	User browses jewelry by category, material, or price.
UC-4	Search Products	User	User searches for specific jewelry using keywords.
UC-5	Add to Cart	User	User adds selected jewelry items to shopping cart.
UC-6	Checkout	User	User completes order with payment and shipping details.
UC-7	Manage Products	Admin	Admin adds, updates, or removes product listings.
UC-8	Manage Orders	Admin	Admin views and updates order statuses.
UC-9	Manage Customers	Admin	Admin views and manages registered users.
UC-10	Logout	User/Admin	User or Admin ends their session securely.

3.2 Functional Requirements

1. User Management

Users can register with a unique email and password. The system validates credentials during login. Admins can view, update, or delete user profiles.

Identifier	Details
Identifier	USR-001
Title	User Registration and Login
Requirement	Users register with unique email/password. System validates login credentials.
Source	Customer Requirements

Identifier	Details
Rationale	Enables secure access and prevents duplicate accounts.
Business Rule	Each email must be unique in the system.
Priority	High

2. Product Management

Admins can add new jewelry products with details (name, price, description, category, stock). Admins can update or delete listings. Users can filter by price, material, and category.

Identifier	Details
Identifier	ADM-001
Title	Product Management
Requirement	Admin adds/updates/deletes jewelry products with full details.
Source	Admin Requirements
Rationale	Maintains an accurate and up-to-date product catalog.
Business Rule	Stock cannot be set below 0.
Priority	High

3. Cart Management

Users can add, update, or remove items from their cart. The system calculates totals including applicable charges. Cart persists across sessions for logged-in users.

4. Order Management

Users complete checkout by providing delivery and payment details. Orders are stored in the database. Admins can view and update order status (Pending, Processing, Shipped, Delivered).

5. Session Management

Secure logout for both users and admins. Automatic session timeout after 15 minutes of inactivity.

3.3 Non-Functional Requirements

- Performance: Pages must load within 3 seconds under normal network conditions.
- Scalability: The system should support at least 100 concurrent users.
- Security: All user passwords are hashed using Django's built-in PBKDF2 algorithm. CSRF protection is enforced on all forms.
- Availability: System should maintain 99% uptime.
- Usability: The interface must be intuitive for non-technical users, with responsive design for mobile and desktop.
- Maintainability: Modular Django app structure allows easy updates and feature additions.
- Compliance: The system follows basic data protection principles and does not store payment card data.

4 Design and Architecture

4.1 System Architecture

The Online Jewelry Store follows Django's Model-View-Template (MVT) architecture:

- Templates (Frontend): HTML/CSS/JavaScript pages served by Django. Handles all user interactions.
- Views (Business Logic): Django views process requests, enforce business rules, and return responses.
- Models (Database): Django ORM models map Python classes to SQLite database tables.
- Admin Panel: Django's built-in admin interface allows administrators to manage data directly.

Module Relationships:

- Templates communicate with Views through HTTP requests.
- Views interact with Models for database operations via Django ORM.
- Django Admin communicates directly with Models for backend management.
- Static files (CSS, JS, images) are served separately via Django's static files system.

4.2 Data Representation [Diagram + Description]

The system's database is structured using SQLite with the following main models:

User Model

Represents registered users.

- user_id (int): Unique identifier.
- username (string): Unique username.

- email (string): Unique email address.
- password (string): Hashed password.
- date_joined (datetime): Account creation date.

Product Model

Represents jewelry products.

- product_id (int): Unique identifier.
- name (string): Product name.
- description (text): Detailed description.
- price (decimal): Product price.
- category (string): e.g., Gold, Silver, Diamond, Fashion.
- stock (int): Available quantity.
- image (ImageField): Product photo.

Order Model

Represents a customer order.

- order_id (int): Unique identifier.
- user (ForeignKey → User): Customer who placed the order.
- total_price (decimal): Total order amount.
- status (string): Pending / Processing / Shipped / Delivered.
- created_at (datetime): Order placement time.

CartItem Model

- cart_id (int): Unique identifier.
- user (ForeignKey → User): Cart owner.
- product (ForeignKey → Product): Product in cart.
- quantity (int): Quantity selected.

[Diagram 2: Data Representation — ER Diagram]

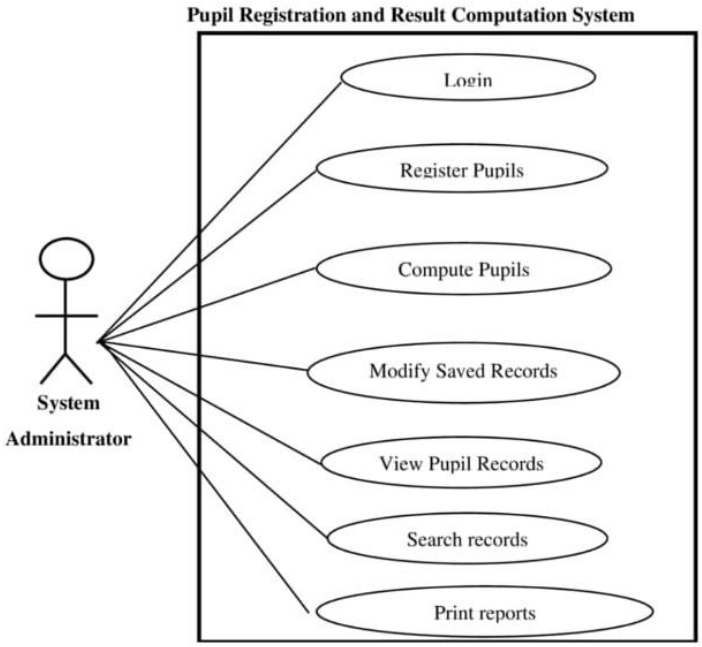
4.3 Process Flow/Representation

The main user flow is: User visits homepage → Browses/searches products → Adds items to cart → Logs in (if not already) → Proceeds to checkout → Enters delivery details → Confirms order → Receives confirmation message.

4.4 Design Models

Sequence Diagram:

- Customer visits the Online Jewelry Store.
- System loads product catalog from the database.
- Customer adds a jewelry item to cart.
- System verifies stock availability.
- Customer proceeds to checkout and provides delivery details.
- System processes the order and updates the database.
- System sends order confirmation to the customer.

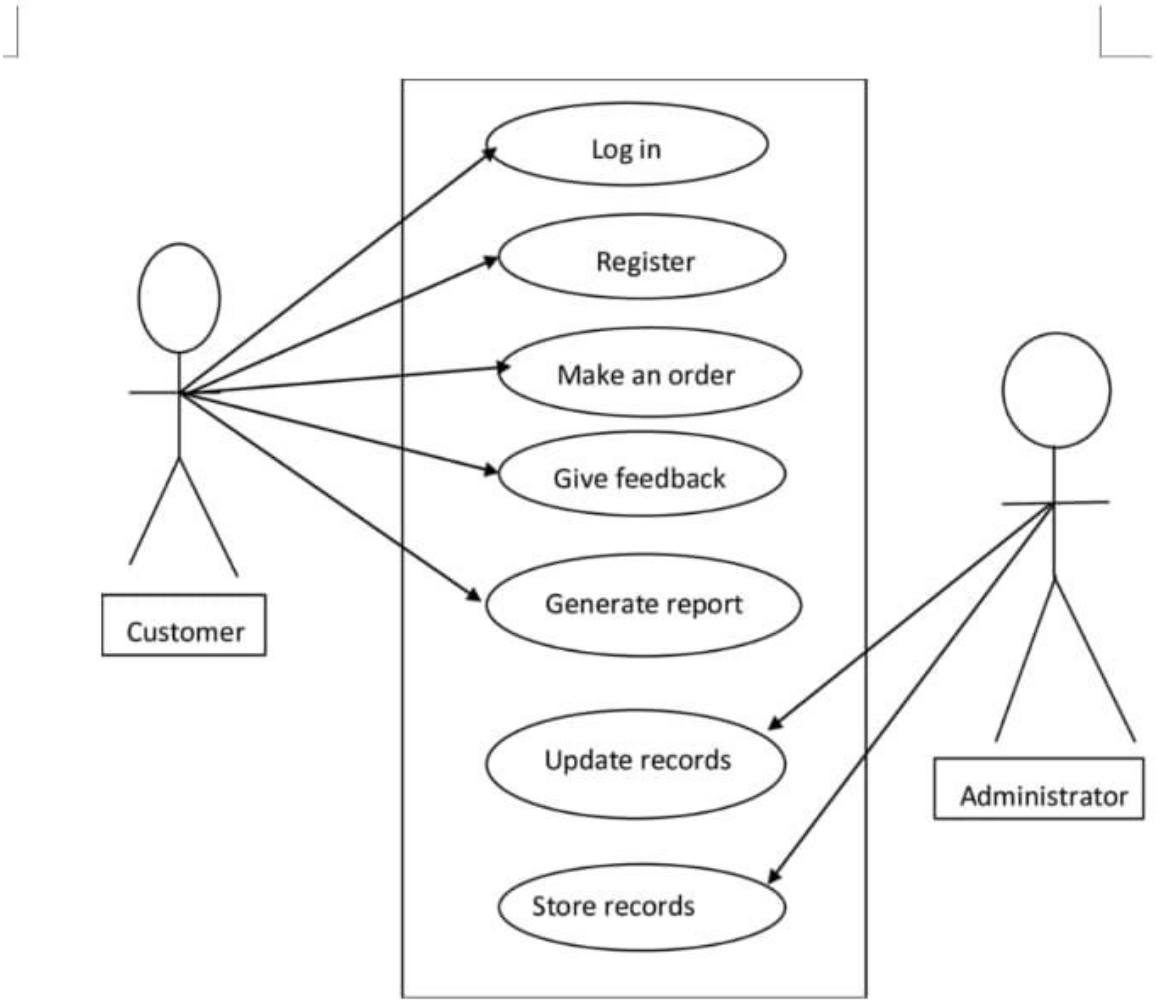


[Diagram 4: Sequence Diagram]

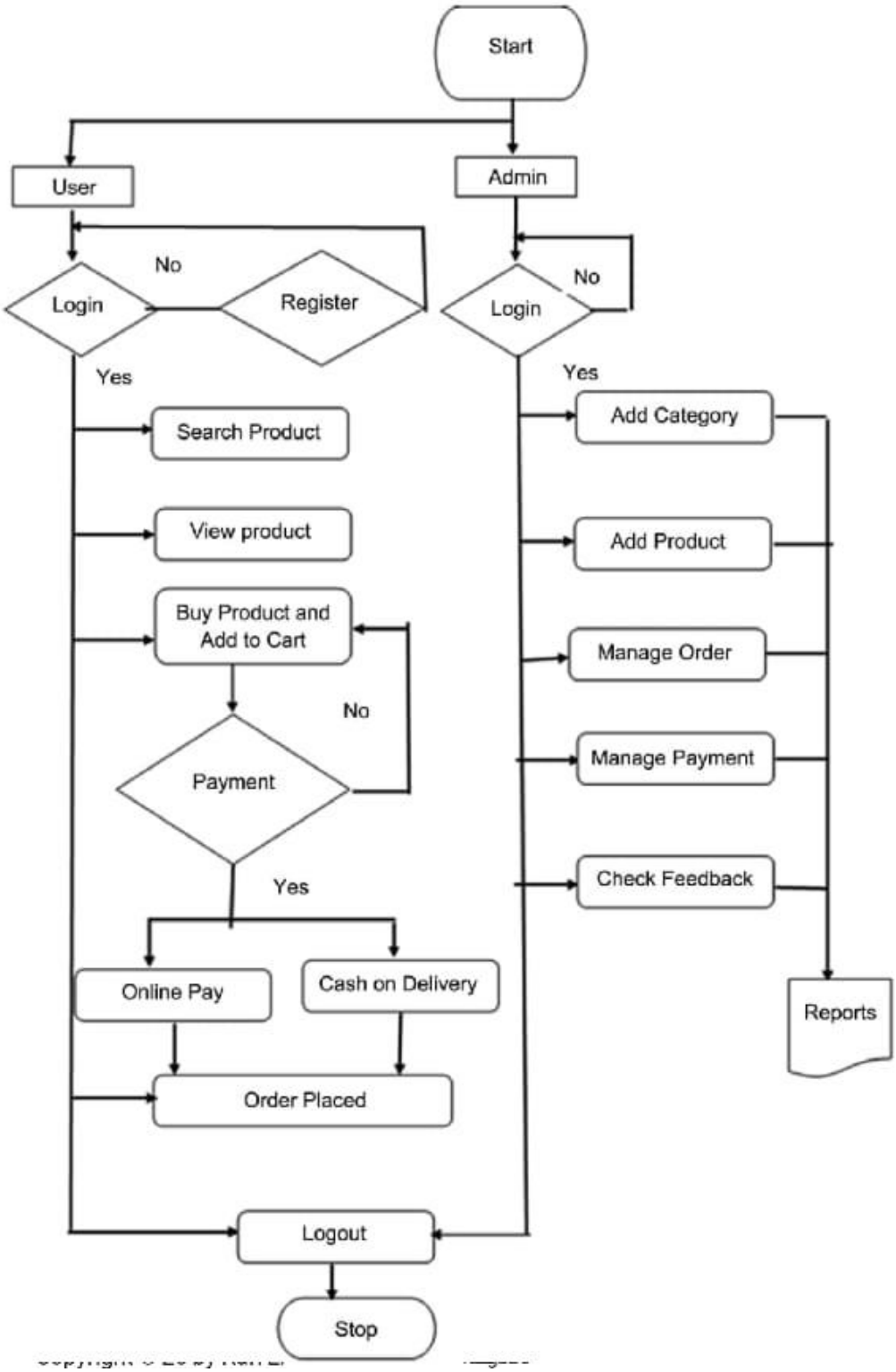
State Transition Diagram:

- States: Not Logged In → Browsing (Guest) → Logged In → Browsing (Logged In) → Cart → Checkout → Order Placed.

- Transitions trigger on user actions such as login, add to cart, and confirm order.



[Diagram 5: State Transition Diagram]



[Diagram 6: Data Flow Diagram]

5 Implementation

The implementation of the Online Jewelry Store follows Django's MVT architecture. Each Django app handles a specific module (products, cart, orders, accounts). The Agile methodology guided iterative development with regular testing and feature refinement.

5.1 Algorithm

5.1.1 User Registration and Authentication Algorithm

Purpose: To securely register new users and authenticate existing users.

Pseudocode:

FUNCTION register_user(username, email, password):

IF email already exists in DB: RETURN error 'Email already registered'

hash = PBKDF2_hash(password)

CREATE User(username, email, hash)

RETURN 'Registration successful'

FUNCTION login_user(email, password):

user = GET User WHERE email = email

IF user NOT FOUND: RETURN 'Invalid credentials'

IF check_password(password, user.hash): CREATE session; RETURN dashboard

ELSE: RETURN 'Invalid credentials'

5.1.2 Product Search and Filter Algorithm

Purpose: Allow users to search and filter jewelry by keyword, category, and price range.

Pseudocode:

FUNCTION search_products(keyword, category, min_price, max_price):

results = Product.objects.filter(name__icontains=keyword)

IF category: results = results.filter(category=category)

IF min_price: results = results.filter(price__gte=min_price)

IF max_price: results = results.filter(price__lte=max_price)

RETURN results

5.1.3 Shopping Cart Management Algorithm

Purpose: Manage adding, updating, and removing products from the cart.

Pseudocode:

FUNCTION add_to_cart(user, product_id, quantity):

product = GET Product WHERE id = product_id

IF product.stock < quantity: RETURN 'Insufficient stock'

cart_item = GET CartItem WHERE user=user AND product=product

IF cart_item EXISTS: cart_item.quantity += quantity

ELSE: CREATE CartItem(user, product, quantity)

SAVE; RETURN 'Added to cart'

5.1.4 Order Placement Algorithm

Purpose: Validate cart and create a confirmed order.

Pseudocode:

FUNCTION place_order(user, delivery_address):

cart_items = GET CartItems WHERE user = user

IF cart_items EMPTY: RETURN 'Cart is empty'

*total = SUM(item.product.price * item.quantity for item in cart_items)*

CREATE Order(user, total, delivery_address, status='Pending')

FOR each item: CREATE OrderItem; UPDATE product.stock -= item.quantity

DELETE CartItems for user; RETURN 'Order placed successfully'

5.1.5 Admin Product Management Algorithm

Purpose: Enable admins to add, update, or delete jewelry products.

Pseudocode:

FUNCTION admin_add_product(name, description, price, category, stock, image):

IF price <= 0 OR stock < 0: RETURN 'Invalid product details'

CREATE Product(name, description, price, category, stock, image)

RETURN 'Product added successfully'

5.2 External APIs

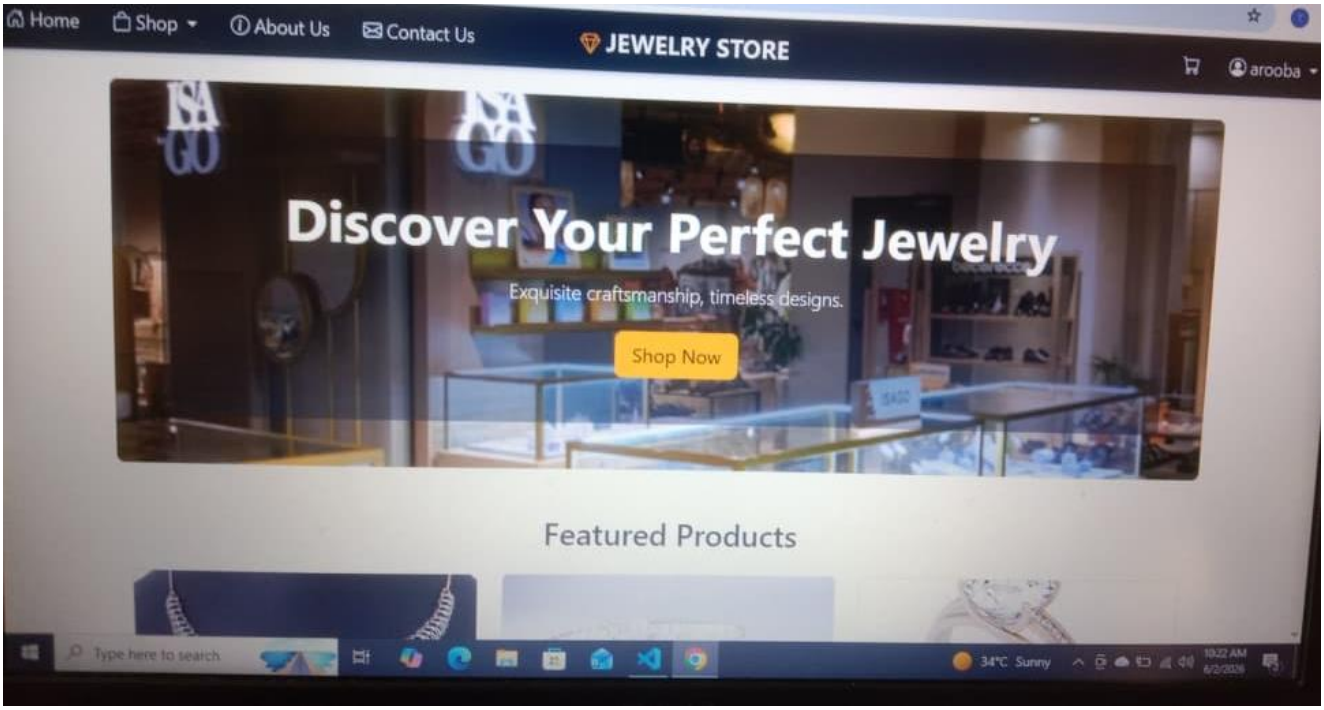
Name of API	Description	Purpose	Function Used
EmailJS / SMTP	API for sending automated emails.	Send order confirmation emails to customers.	send_order_confirmation()
Cloudinary (optional)	Cloud-based image storage API.	Store and serve product images efficiently.	upload_product_image()

5.3 User Interface

The UI of the Online Jewelry Store is designed to be clean, elegant, and responsive — reflecting the premium nature of jewelry products. Key interface components include:

5.3.1 Home Page

The Home Page features a fixed navigation bar with links to Home, Products, Categories, and Contact. A prominent search bar allows keyword-based product search. A hero section showcases featured jewelry with a call-to-action button. Product highlights and categories are displayed in a responsive grid layout.



[Figure 2: Home Page Interface]

5.3.2 Django Admin Interface

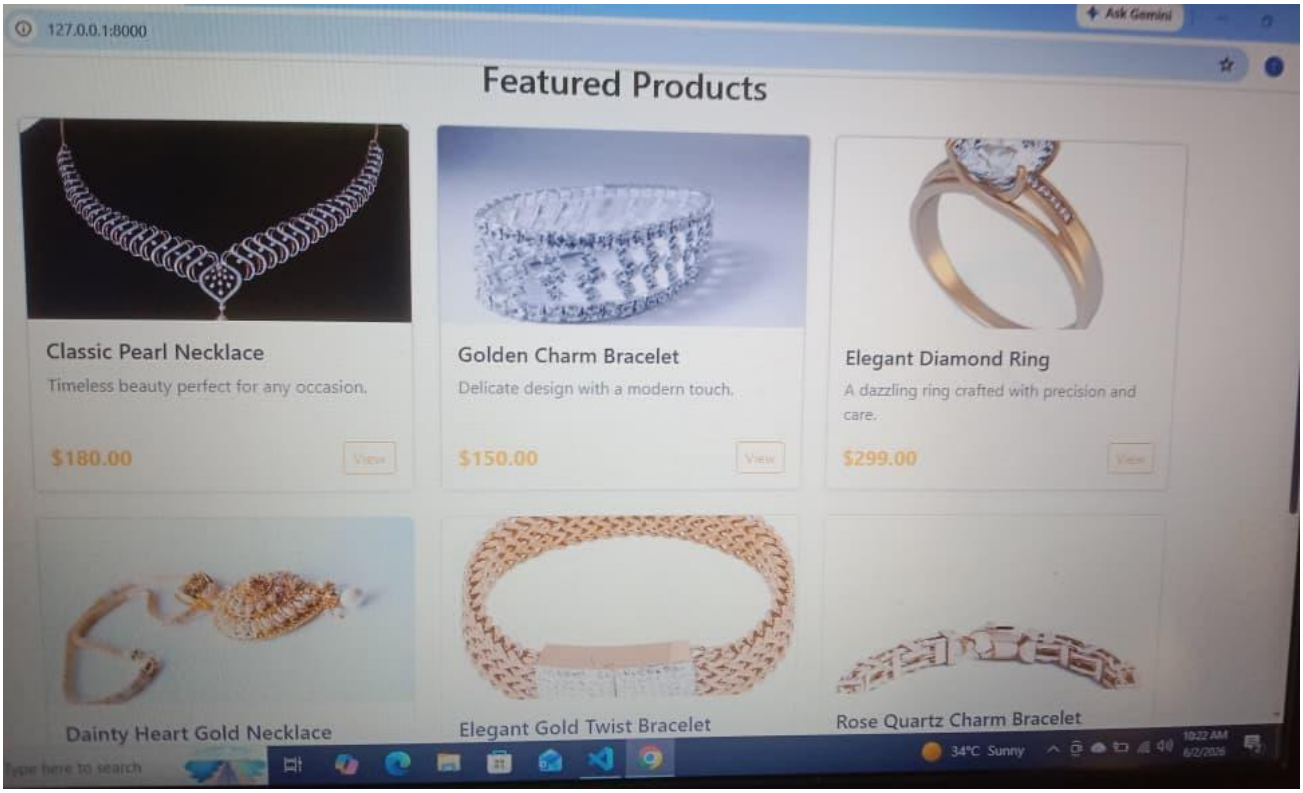
The Django Admin Panel is used to manage the database. It provides a clean interface to add, edit, or delete Products, Orders, and Users. The admin can update order statuses and monitor inventory levels directly from this panel.

5.3.3 Login Page Interface

The Login Page offers a simple, centered form with fields for email and password. It includes client-side validation and links to the registration page for new users. Django's CSRF protection is enforced on the form.

5.3.4 Product Listing & Detail Page

Products are displayed in a card-based grid layout with images, names, categories, and prices. Clicking a product opens a detail page with full description, stock status, and an Add to Cart button.



[Figure 5: Product Listing Page]

6 Testing and Evaluation

Testing was performed both manually and using automated tools to ensure the system functions correctly, securely, and efficiently.

6.1 Manual Testing

6.1.1 System Testing

System testing was performed after full integration to verify the complete application works as intended. This ensured all modules (authentication, product management, cart, orders) work correctly together.

6.1.2 Unit Testing

Unit Testing 1: User Login

Testing Objective: To ensure the login form works correctly with valid and invalid inputs.

No.	Test Case	Input	Expected Result	Result
1	Login with correct credentials	Email: arooba@gmail.com Password: Arooba123	Successfully logs in and redirects to home page.	Pass

No.	Test Case	Input	Expected Result	Result
2	Login with wrong password	Email: arooba@gmail.com Password: wrongpass	Error: Incorrect email or password.	Pass
3	Login with unregistered email	Email: unknown@gmail.com	Error: Account not found.	Pass

Unit Testing 2: Add Product (Admin)

Testing Objective: To ensure admin can add jewelry products correctly.

No.	Test Case	Input	Expected Result	Result
1	Add valid product	Name, Price, Stock, Image provided	Product added and displayed in catalog.	Pass
2	Add product with negative price	Price: -500	Validation error shown.	Pass

6.1.3 Functional Testing

Functional Testing 1: User Shopping Flow

Objective: To verify the complete flow from product browsing to order placement.

No.	Test Case	Input	Expected Result	Result
1	Browse and add to cart	Select a jewelry item	Item appears in cart with correct price.	Pass
2	Checkout	Delivery address + confirm	Order created, cart cleared, confirmation shown.	Pass
3	Admin view order	Admin checks order list	New order visible with status 'Pending'.	Pass

6.1.4 Integration Testing

No.	Test Case	Attribute and Value	Expected Result	Result

No.	Test Case	Attribute and Value	Expected Result	Result
1	User logs in and views cart	Valid credentials	Cart loads with correct user's items.	Pass
2	Admin updates order status	Order ID + status change	Order status updated in database and visible to user.	Pass
3	Product stock decreases after order	Order placed for 2 items	Product stock reduced by 2.	Pass

6.2 Automated Testing

Tool Name	Description	Applied On	Results
Django Test Framework	Built-in Python unit testing for Django views and models.	User registration, login, cart, order placement (FR-001 to FR-005)	Pass
Selenium IDE	Browser automation tool for UI testing.	Login form, product search, add to cart, checkout flow	Pass
Postman	API testing for Django REST endpoints.	Product listing API, order status API, user auth API	Pass

7 Conclusion and Future Work

7.1 Conclusion

This project successfully delivered a fully functional Online Jewelry Store built with HTML, CSS, JavaScript, Python (Django), and SQLite. The system provides a seamless shopping experience for customers and an efficient management system for administrators.

Key Achievements:

- Developed a complete e-commerce platform with user authentication, product catalog, cart management, and order processing.
- Implemented Django's MVT architecture for a clean, maintainable, and scalable codebase.
- Created a secure backend with CSRF protection, password hashing, and session management.
- Built a responsive, mobile-friendly frontend using HTML, CSS, and JavaScript.
- Used SQLite as a lightweight, serverless database suitable for development and small-scale deployment.
- Completed thorough manual and automated testing to ensure reliability and correctness.

7.2 Future Work

While this project lays a solid foundation, the following enhancements are planned for future versions:

- Payment Gateway Integration: Add support for JazzCash, Easypaisa, and Stripe for real online payments.
- Product Recommendations: Implement an AI-based recommendation engine using customer browsing history.
- Mobile App: Develop a dedicated Android/iOS app using Django REST Framework and React Native.
- Advanced Search: Add voice search and image-based product search capabilities.
- Inventory Alerts: Automated email/SMS notifications to admin when stock falls below a threshold.
- Multi-language Support: Add Urdu language support for wider accessibility in Pakistan.
- Customer Reviews: Enable product ratings and reviews to build trust and assist purchasing decisions.
- Cloud Deployment: Migrate from SQLite to PostgreSQL and deploy on AWS or DigitalOcean for production.

References

- Django Documentation — https://docs.djangoproject.com
- Python Documentation — https://docs.python.org
- HTML & CSS Reference — https://www.w3schools.com
- JavaScript Reference — https://developer.mozilla.org
- Bootstrap Documentation — https://getbootstrap.com/docs
- SQLite Documentation — https://www.sqlite.org/docs.html
- OWASP Security Guidelines — https://owasp.org
- Pakistan E-Commerce Industry Report 2024 — PASHA
- Agile Manifesto — https://agilemanifesto.org